

CampusMind: BFS, DFS and A* Compared

ICS 2413 Artificial Intelligence. Part 3 (Week 3). Owen Monda

1. Route chosen

Main Gate (L1) to Science Laboratory (L4). I chose it because the map gives two clearly different ways to do it. The short way goes through the Administration Block ($1 + 2 = 3$ km). The long way goes round the loop through the Cafeteria and Student Affairs ($3 + 2 + 2 = 7$ km). A good algorithm should find the 3 km route.

2. Results on this route

Algorithm	Path found	Cost	Hops	Locations expanded
BFS	Main Gate, Administration Block, Science Lab	3 km	2	5
DFS (default order)	Main Gate, Administration Block, Science Lab	3 km	2	4
DFS (reversed order)	Main Gate, Cafeteria, Student Affairs, Science Lab	7 km	3	4
A*	Main Gate, Administration Block, Science Lab	3 km	2	3

3. Search behavior of each algorithm

BFS keeps its frontier as a queue: first in, first out. It expands the Main Gate, then everything one connection away (Administration Block, Cafeteria), then everything two away (Library, Science Lab). It finishes each layer before starting the next, so it always finds a route with the fewest connections. It expanded 5 locations before popping the Science Lab.

DFS keeps its frontier as a stack: last in, first out. It picks one neighbour and keeps going deeper before it ever tries the other. In the default order it went to the Administration Block, tried the Library (a dead end), backed up, and reached the Science Lab. It expanded 4 locations.

A* keeps its frontier as a priority queue ordered by $f = g + h$, where g is the real cost so far and h is the heuristic guess of what is left. It always expands the entry with the lowest f . It expanded the Main Gate, the Administration Block and the Science Lab, which is 3 locations.

4. Heuristic used and why it is admissible

I used **Option A, the hop-count heuristic**. $h(\text{place})$ is the fewest connections needed to reach the goal, ignoring distances. For the Science Lab as goal:

Science Lab 0, Administration Block 1, Student Affairs 1, Main Gate 2, Library 2, Cafeteria 2

Admissibility argument. Every connection costs at least 1 km, so a route with k connections costs at least k km, which means the true cost can never be smaller than the hop count, so h never overestimates. I also checked it by code against the true shortest costs for all six goals and no estimate was too high. It is also consistent, because moving across one connection lowers h by at most 1 while costing at least 1, and that is why A* can safely skip a location once it has been expanded.

What it estimates. It says how many more steps are needed at minimum. It is deliberately optimistic, which is what keeps A* honest.

5. A* step by step

Step 1: frontier is Main Gate ($f = 0 + 2 = 2$). Expand it, adding Administration Block (g 1, h 1, f 2) and Cafeteria (g 3, h 2, f 5).

Step 2: lowest f is the Administration Block (2). Expand it. Main Gate is already done. Add Science Lab (g 3, h 0, f 3) and Library (g 3, h 2, f 5).

Step 3: lowest f is now the Science Lab (3), lower than both $f = 5$ entries. It is popped and it is the goal, so the search ends with cost 3. The goal is detected when it is taken off the frontier, not when it is first added, because only then is no cheaper route still hiding in the queue.

6. Comparison of the three

All three found the 3 km route in the default setup, so on this single route the difference is in the work done: A* 3 expansions, DFS 4, BFS 5. That gap is small because the map only has six places. To check that this was not luck I ran every start and goal pair (30 pairs):

Algorithm	Optimal routes found (of 30)	Total expansions
BFS	30	120
DFS (default order)	18	120
A*	30	96

BFS was optimal every time, but that is a feature of this particular map. Here the route with the fewest connections always happens to be the cheapest too. BFS does not look at kilometres at all, so on a map where a many-hop route is shorter than a few-hop route, it would return the wrong answer. DFS is the least reliable. Its answer depends on the order neighbours are listed. Reversing that order on the same route gave the 7 km path, and over all 30 pairs it missed the best route 12 times. A* was optimal every time and did the least work.

Which I would prefer. A* when I need the cheapest route and can write an honest heuristic. BFS when every step costs the same and I only want the fewest steps. DFS when I only need any route, or when memory is tight, since a stack stays small.

7. Why A* performed differently

A* never expanded the Cafeteria or the Library. Their f value was 5, higher than the Science Lab's f of 3, so the priority queue always chose something better first. BFS expanded both because it ignores everything except how many layers away a place is. DFS avoided the Cafeteria only because of the neighbour order, which is luck. A* avoided it by reasoning: g told it what the route had cost, and h told it the Cafeteria was still 2 connections from the goal, so that branch looked worse. On this map the saving is modest (96 versus 120 expansions across all pairs). The heuristic is loose, since it treats a 3 km connection like a 1 km one, so it guides but does not narrow the search as sharply as a distance-based estimate would.